

$$\theta_{k+1} \leftarrow \theta_k - \eta \nabla f(\theta_k)$$

η = función de aprendizaje

$$f_{\theta}(x) = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = w^T x + b = [x^T, 1] \begin{bmatrix} w \\ b \end{bmatrix} = [x^T, 1] \theta$$

$$\theta^* = w^*, b^* = \arg \min_{\theta \in \mathbb{R}^{n+1}} \frac{1}{2M} \sum_{i=1}^M (\gamma^{(i)} - [x^T, 1] \theta)^2$$

para w_j

$$\frac{\partial E_n(\theta)}{\partial w_j} = \frac{1}{2M} \sum_{i=1}^M -2(\gamma^{(i)} - \hat{\gamma}^{(i)}) x_j^{(i)}$$

γ = constante

$\hat{\gamma} = \gamma$ estimada

$$\frac{\partial E_n(\theta)}{\partial b} = \frac{1}{2M} \sum_{i=1}^M -2(\gamma^{(i)} - \hat{\gamma}^{(i)})$$

$$\nabla E_n(\theta) = -\frac{1}{M} \begin{bmatrix} \sum_{i=1}^M (\gamma^{(i)} - \hat{\gamma}^{(i)}) x_1^{(i)} \\ \sum_{i=1}^M (\gamma^{(i)} - \hat{\gamma}^{(i)}) x_2^{(i)} \\ \vdots \\ \sum_{i=1}^M (\gamma^{(i)} - \hat{\gamma}^{(i)}) x_n^{(i)} \\ \sum_{i=1}^M (\gamma^{(i)} - \hat{\gamma}^{(i)}) \end{bmatrix}$$

$\nabla E_n(\theta)$

El gradiente de la función de costo evaluado en θ

$$= \begin{bmatrix} x_e^{(1)} & x_e^{(2)} & \dots & x_e^{(m)} \\ x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(m)} \\ \vdots & \vdots & & \vdots \\ x_n^{(1)} & x_n^{(2)} & \dots & x_n^{(m)} \\ 1 & 1 & \dots & 1 \end{bmatrix} \begin{bmatrix} \gamma^{(1)} - \hat{\gamma}^{(1)} \\ \gamma^{(2)} - \hat{\gamma}^{(2)} \\ \vdots \\ \gamma^{(m)} - \hat{\gamma}^{(m)} \end{bmatrix}$$

x_e^T $\gamma - x_e \theta$
 $\uparrow$ $\uparrow$
 $x_w + b$

$$\theta \leftarrow \theta - \eta \nabla E_n(\theta)$$

$$\theta \leftarrow \theta + \frac{\eta}{M} x_e^T (\gamma - x_e \theta)$$

funcion de aprendizaje

lr = learning rate e_tol = error de tolerancia

```
def dg_lin(X, Y, w0, b0, lr, max_epochs, e_tol):  
    m = X.shape[0]
```

```
    w = w0.copy()
```

```
    b = b0.copy()
```

```
    hist = []
```

```
    for _ in range(max_epochs):
```

```
        Y_est = X @ w + b
```

```
        Err = Y - Y_est
```

```
        hist.append(np.square(Err).mean())
```

```
        grad_w = -(1/m) X.T @ Err
```

```
        d_b = Err.mean()
```

```
        w -= lr * grad_w
```

```
        b -= lr * d_b
```

```
        if np.abs(grad_w).max() < e_tol:  
            break
```

```
    return w, b, hist
```

Ejemplo

X, Y

X.shape = [m, n], Y.shape = [m,]

w = np.zeros(X.shape[-1])

b = 0

w_n, b_n, hist = dg_lin(X, Y, w, b, 0.1, 50, -1.e-4)